

ChordiNote

An Easy Project, Algorithm, Presentation, and Viva Guide

Study notes for project presentation

Contents

1	The Project in One Minute	2
2	Why Is Music Transcription Difficult?	2
3	The Whole ChordiNote Pipeline	2
3.1	A simple pipeline diagram	3
4	Algorithms Explained Easily	3
4.1	1. CQT: Constant-Q Transform	3
4.2	2. NMF: Non-negative Matrix Factorization	4
4.3	3. Note Templates and Harmonics	4
4.4	4. Onset Detection: Finding When a Note Starts	4
4.5	5. pYIN: Estimating a Monophonic Pitch	5
4.6	6. HMM and Viterbi: Smoothing Note Activity	5
4.7	7. Converting Frequency to MIDI Notes	5
4.8	8. Filtering Harmonic Ghost Notes	6
4.9	9. Musical and Instrument Constraints	6
4.10	10. Chord Grouping, Rests, and Audio Reconstruction	6
5	A Short Worked Example	6
6	Sample Presentation Script	7
7	Viva Questions and Short Answers	8
7.1	Project and pipeline	8
7.2	CQT and NMF	8
7.3	Onsets, pYIN, and HMM	8
7.4	Music theory, output, and limitations	9
8	Last-Minute Memory Sheet	10

1 The Project in One Minute

ChordiNote is an automatic music transcription system. It takes an audio recording—for example, piano, guitar, flute, or a vocal melody—and tries to identify the musical notes, their timing, and notes that occur together as chords.

A recording does not directly contain labels such as C_4 , E_4 , or G_4 . It contains a changing sound wave. ChordiNote analyzes that sound and converts it into a musical description.

Main idea: audio waveform $\rightarrow$ frequency information $\rightarrow$ note candidates $\rightarrow$ cleaned note events $\rightarrow$ readable transcription.

The project can produce timestamped notes/chords, visualizations such as a keyboard or guitar fretboard, and synthesized audio reconstructed from the detected notes. The exact result depends on the recording quality, instrument, and how clearly the notes are separated.

2 Why Is Music Transcription Difficult?

There are several reasons this task is not just “find the loudest frequency.”

1. **Multiple notes can overlap.** A piano chord may contain several notes at the same time, and their frequency components overlap.
2. **One note contains harmonics.** A note with fundamental frequency f_0 can also contain energy near $2f_0$, $3f_0$, $4f_0$, and so on. These harmonics can look like additional notes.
3. **Notes have beginnings and endings.** A repeated piano note may start again before the previous sound has fully faded.
4. **Real instruments are not ideal sine waves.** Instrument timbre and, for piano strings, inharmonicity can make real spectra differ from simple templates.
5. **Recordings may contain silence, noise, or weak notes.** A system must avoid turning every tiny fluctuation into a musical note.

The project therefore combines signal-processing algorithms with musical and instrument-specific constraints.

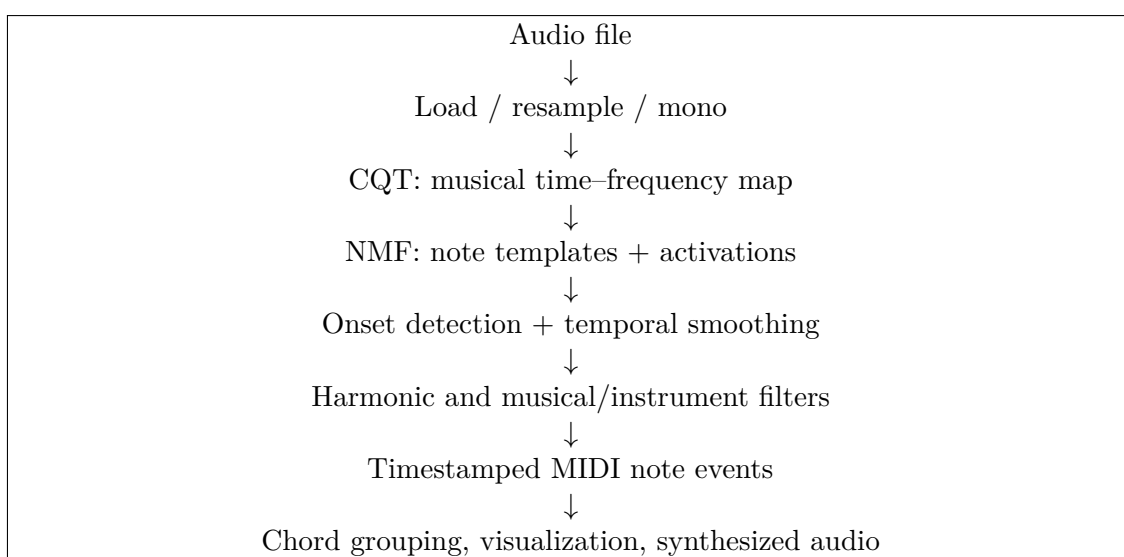
3 The Whole ChordiNote Pipeline

The following is the overall idea in sequence. Some stages are specialized: pYIN is mainly for monophonic pitch tracking, while NMF-based engines address polyphonic note detection.

1. **Load audio.** Read the recording, resample it to the project’s working rate (22,050 Hz), and convert it to mono for analysis.
2. **Analyze time and frequency.** Compute a time-frequency representation. CQT is especially useful because its frequency bins are logarithmically spaced like musical pitches.
3. **Build note templates.** Create a frequency-pattern template for each candidate note. Templates can account for harmonics and piano-string inharmonicity.
4. **Estimate note activity.** NMF compares the observed CQT representation with the templates. Its activation matrix estimates which notes are present at each moment.
5. **Find note attacks.** Energy changes and spectral flux help locate onsets, including repeated notes.

6. **Stabilize the estimates.** HMM/Viterbi smoothing can turn noisy frame-by-frame activations into more consistent ON/OFF note tracks. pYIN uses a related probabilistic sequence stage for monophonic pitch tracking.
7. **Remove likely false detections.** Apply harmonic-ghost checks, pitch-aware thresholds, polyphony limits, scale/key context, and instrument constraints.
8. **Create note events.** Convert active regions into notes with start/end times and MIDI pitches. Use onset positions to split repeated notes.
9. **Group and present.** Group overlapping notes into chords, represent gaps as rests where appropriate, and display the transcription.
10. **Reconstruct and evaluate.** Synthesize audio from the note events and compare outputs using the project's evaluation measures.

3.1 A simple pipeline diagram



4 Algorithms Explained Easily

4.1 1. CQT: Constant-Q Transform

Problem it solves: How can we represent sound frequencies in a way that is convenient for musical notes?

A waveform shows amplitude changing over time. It does not directly show which musical pitches are present. A time-frequency transform gives a picture of frequency content across time.

CQT uses **logarithmically spaced frequency bins**. This matches musical pitch: going up one octave doubles frequency. For example, A4 is 440 Hz and A5 is 880 Hz, but they are the same note name in different octaves.

A common CQT frequency-bin formula is

$$f_k = f_{\min} 2^{k/B},$$

where $f_{\min}$ is the starting frequency, k is a bin index, and B is the number of bins per octave.

In this project, $B = 36$: there are 36 bins per octave, or 3 bins per semitone because an octave contains 12 semitones.

Intuition: CQT is like a piano-key-organized frequency map. It helps the later algorithms compare audio against musical note patterns.

Why not only ordinary FFT? An ordinary FFT has equally spaced frequency bins. CQT's logarithmic spacing is often more convenient when analyzing musical pitch over several octaves.

4.2 2. NMF: Non-negative Matrix Factorization

Problem it solves: How can we estimate several notes that are mixed together in one recording?

After CQT, the audio is represented as a non-negative time-frequency matrix V . NMF approximates it as

$$V \approx WH$$

where:

- V is the observed CQT magnitude representation.
- W contains note templates. Each column is a frequency-pattern “fingerprint” for a candidate note.
- H contains activations. Each value indicates how strongly a note template is active at a particular time frame.

Analogy: Imagine a song as a mixture of colored paints. W contains the available colors, and H estimates how much of each color is needed at each moment to reproduce the mixture. Here, the “colors” are note spectra.

For a C-major chord, the activation rows corresponding to C, E, and G may all be strong during the same interval.

The project uses iterative non-negative updates to improve the estimate of H . A simplified multiplicative update is

$$H \leftarrow H \odot \frac{W^T V}{W^T W H + \epsilon},$$

where $\odot$ and the division are element-wise, and ϵ prevents division by zero.

Important limitation: NMF does not magically know the correct notes. If templates are inaccurate or harmonics resemble other notes, it can produce false activations. Later filtering is needed.

4.3 3. Note Templates and Harmonics

A musical tone is usually not a single frequency. If its fundamental is f_0 , its harmonic components may occur near

$$f_0, 2f_0, 3f_0, 4f_0, \dots$$

These components help give an instrument its timbre.

For example, a note with a 200 Hz fundamental may also contain energy near 400 Hz and 600 Hz. A naive detector could mistake 400 Hz for a separate note.

ChordNote’s templates model harmonic structure. For piano, the report describes an inharmonicity-aware Fletcher-style relation:

$$f_n = n f_0 \sqrt{1 + B n^2},$$

where n is the harmonic number and B is a small inharmonicity coefficient (approximately 2×10^{-4} in the described setup).

This helps templates better resemble real piano-string spectra instead of assuming perfectly harmonic frequencies.

4.4 4. Onset Detection: Finding When a Note Starts

An **onset** is the beginning or attack of a musical event. It is not necessarily the moment when all previous sound has become silent.

Why do we need it? If a piano key is struck twice quickly, the sound from the first strike may still be ringing when the second strike occurs. Onset detection helps identify the second attack and split the notes correctly.

The project combines two kinds of evidence:

- **Energy-based detection:** examines changes in signal energy, such as a sudden rise in RMS energy.
- **Spectral flux:** examines how much the spectrum changes between adjacent frames. A new attack often changes the frequency content.

Using both can be more robust than relying on only one signal feature. The detected onsets help split repeated notes and support note/chord event organization.

4.5 5. pYIN: Estimating a Monophonic Pitch

Problem it solves: For a signal with one main note at a time, what pitch is being sung or played? pYIN estimates the fundamental frequency, often written F_0 . The YIN method looks for periodicity in the waveform: it compares the signal with shifted versions of itself and searches for a likely repeating period T .

Once a period is estimated,

$$F_0 = \frac{1}{T}$$

For example, if the period is about $1/440$ seconds, the estimated fundamental is about 440 Hz, corresponding to A4.

The “p” in pYIN refers to its probabilistic approach. Rather than treating each frame’s pitch estimate as completely independent, pYIN considers possible pitch candidates and uses an HMM/Viterbi sequence stage to produce a more consistent pitch track and reduce octave errors.

Scope note: pYIN is mainly suited to monophonic pitch tracking. A full piano chord contains multiple simultaneous fundamentals, so the project’s NMF-based polyphonic engines are used for that problem.

4.6 6. HMM and Viterbi: Smoothing Note Activity

An HMM, or Hidden Markov Model, represents a sequence using hidden states and observed evidence.

For each candidate note in the NMF stage, the project’s simplified state idea is:

OFF or ON.

The NMF activation is the observation: a high activation supports ON, while a low activation supports OFF. The model also considers transition probabilities. For example, a note that is ON in one frame is often likely to remain ON in the next frame.

This helps prevent a note track from flickering because of one weak frame.

Viterbi decoding finds the most likely sequence of hidden states given the observations and transition probabilities.

Easy distinction:

- NMF estimates note strength frame by frame.
- HMM/Viterbi turns those noisy strengths into a more consistent ON/OFF sequence.

4.7 7. Converting Frequency to MIDI Notes

Once a frequency estimate is available, it can be mapped to a standard MIDI pitch number:

$$m = 69 + 12 \log_2 \left(\frac{f}{440} \right)$$

Here f is frequency in Hz, and MIDI 69 corresponds to A4 = 440 Hz. The result is rounded to the nearest integer MIDI note when a discrete note label is needed.

MIDI numbers make it easy to represent notes consistently, compare pitches, and render keyboard visualizations.

4.8 8. Filtering Harmonic Ghost Notes

A **ghost note** is a false note candidate that appears because another note's harmonics or spectral structure activated its template.

The system checks whether a candidate frequency is close to an integer multiple of a lower note's frequency. It also uses attack/onset evidence: a true independently played note may have its own attack, while a harmonic component may not.

The described V1 logic, for example, checks harmonic ratios and compares activations; later versions add onset-guided filtering and improved templates. These are heuristics, so they reduce likely false detections rather than guaranteeing perfect separation.

4.9 9. Musical and Instrument Constraints

After initial note detection, ChordiNote applies additional rules to reduce implausible outputs.

- **Pitch-aware thresholds:** different pitch ranges can require different activation thresholds.
- **Polyphony limits:** limit how many notes are active at once according to the selected instrument profile.
- **Key/scale filtering:** estimate a tonal center using pitch-class profiles and remove short, unlikely out-of-scale artifacts, while allowing chromatic notes when appropriate.
- **Semitone adjacency check:** adjacent semitone candidates can be suspicious; relative activation is used to decide whether to suppress a weaker candidate or retain both.
- **Guitar playability:** check whether a note fits the standard string tunings and available fret range. Competing notes assigned to the same string may be resolved.

These rules add musical or physical context. They should be understood as useful constraints, not universal laws: real music can contain chromatic notes, unusual voicings, and exceptions.

4.10 10. Chord Grouping, Rests, and Audio Reconstruction

After note events have start times, end times, and MIDI pitches, notes with overlapping active intervals can be grouped into a chord. The project groups notes when their overlap is at least about 50 ms.

Gaps can be represented as rests. The output can then be shown as timestamped notes, chord labels, and keyboard/fretboard visualizations.

For audio reconstruction, the project uses additive synthesis: it generates sine-wave components for the detected notes, combines them, and applies an amplitude envelope such as attack, decay, sustain, and release (ADSR). This gives an audible rendering of the transcription, though it will not necessarily sound identical to the original instrument.

5 A Short Worked Example

Suppose a clean monophonic recording contains an A4 note.

1. The waveform is loaded and prepared for analysis.
2. CQT represents its frequency energy in musical frequency bins.
3. pYIN estimates a repeating period of approximately $T = 1/440$ seconds.
4. The estimated fundamental is $F_0 = 1/T \approx 440$ Hz.
5. Frequency is converted to MIDI:

$$m = 69 + 12 \log_2(440/440) = 69.$$

6. MIDI 69 corresponds to A4.
7. The system uses the active time region and onset/end information to form a timestamped note event.

For a polyphonic C-major chord, the NMF stage instead looks for simultaneous activations of the C, E, and G templates. The event stage can group the overlapping notes into a chord.

6 Sample Presentation Script

The following is a speakable script. It is intentionally written in straightforward language. Adjust “we” or implementation details to match what your team actually built and demonstrated.

Opening and project goal

Good morning, Sir. Our project is called ChordiNote. It is an automatic music transcription system. It takes an audio recording, such as piano, guitar, flute, or a vocal melody, and tries to identify the musical notes being played, when they start and end, and which notes occur together as chords.

The audio file does not contain note names directly. It contains a changing sound wave. Our system analyzes that wave and converts it into a musical representation.

Pipeline overview

First, we load and prepare the audio. Then we use CQT to convert it into a time-frequency representation whose frequency bins are arranged logarithmically, matching musical pitch.

Next, NMF compares the observed representation with note templates. The templates describe what candidate notes look like in frequency, and the activation matrix estimates when each note is present.

We use onset detection to find note beginnings, especially when a note is repeated before the previous sound has faded. Temporal smoothing helps reduce unstable note detections. Then we apply harmonic, musical, and instrument-specific filters to remove likely false notes.

Finally, we create timestamped MIDI note events, group overlapping notes into chords, display the result, and synthesize audio from the detected notes.

Explain the core algorithms

CQT is useful because musical pitch is logarithmic: doubling frequency means moving up one octave. NMF is useful because a chord is a mixture of note spectra. It approximates the observed matrix as $V \approx WH$, where W contains note templates and H contains their activations.

Onset detection finds the beginnings of notes using energy changes and spectral changes. For monophonic audio, pYIN estimates the fundamental frequency by detecting waveform periodicity. It then uses probabilistic sequence decoding to make the pitch track more consistent.

For polyphonic note activity, an HMM can model each note as ON or OFF. Viterbi decoding finds the most likely sequence, reducing brief flickers. We then use harmonic relationships and musical/instrument constraints to remove likely false detections.

Closing

In summary, no single algorithm solves the whole transcription problem. CQT represents the audio in a pitch-friendly way, NMF estimates note mixtures, onset detection locates attacks, pYIN tracks monophonic pitch, HMM smoothing improves temporal consistency, and musical constraints reduce implausible detections. Combining these stages gives us a more useful transcription. Thank you, Sir.

7 Viva Questions and Short Answers

7.1 Project and pipeline

1. **What is automatic music transcription?**

Converting audio into musical information such as note pitches, start/end times, and chords.

2. **Why is transcription difficult?**

Notes overlap, each note has harmonics, note attacks can be close together, and real recordings contain noise and instrument-specific timbre.

3. **Why not just select the loudest frequency?**

The strongest component may be a harmonic rather than the fundamental, and polyphonic audio contains several notes at once.

4. **What does each main stage do?**

CQT represents frequencies in a musical layout; NMF estimates note activations; onset detection finds note beginnings; pYIN tracks monophonic pitch; HMM smooths activity; filters remove likely false notes.

7.2 CQT and NMF

5. **What is CQT?**

A time-frequency transform with logarithmically spaced frequency bins, useful for musical pitch analysis.

6. **Why logarithmic bins?**

Musical intervals are logarithmic in frequency. An octave is a frequency doubling.

7. **What does 36 bins per octave mean?**

One octave is divided into 36 bins. Since there are 12 semitones per octave, that is 3 bins per semitone.

8. **What is NMF?**

A method that approximates a non-negative matrix as the product of two non-negative matrices.

9. **Explain $V \approx WH$.**

V is the observed CQT matrix, W contains note templates, and H contains note activation strengths over time.

10. **How can NMF identify a chord?**

Several note-template activations can be strong at the same time. Overlapping note events can then be grouped as a chord.

11. **Does NMF always identify the correct notes?**

No. Template mismatch and harmonic overlap can cause false activations, so the system uses additional filters.

7.3 Onsets, pYIN, and HMM

12. **What is an onset?**

The beginning or attack of a musical event.

13. **What is spectral flux?**

A measure of how much the spectrum changes between consecutive frames. A sudden change can indicate a note attack.

14. Why use energy and spectral flux together?

Energy measures signal-strength changes, while spectral flux measures frequency-content changes. They provide complementary onset evidence.

15. What does pYIN estimate?

The fundamental frequency or pitch track of a mainly monophonic signal.

16. How does YIN estimate pitch?

It searches for waveform periodicity, estimates period T , and calculates $F_0 = 1/T$.

17. Why is pYIN probabilistic?

It considers pitch candidates and temporal consistency rather than making an isolated decision for every frame.

18. Can pYIN handle a full piano chord?

It is mainly intended for monophonic pitch tracking. Polyphonic note detection in this project uses NMF-based engines.

19. What are the HMM states for note activity?

ON and OFF.

20. What does Viterbi do?

It finds the most likely sequence of hidden states based on observations and transition probabilities.

21. Why use HMM after NMF?

To reduce brief activation flickers and produce more consistent note activity over time.

7.4 Music theory, output, and limitations

22. What are harmonics?

Frequency components near integer multiples of a note's fundamental frequency.

23. What is a harmonic ghost note?

A false note candidate caused by energy from another note's harmonic structure.

24. How does the system reduce harmonic ghosts?

It checks integer-multiple frequency relationships and whether the candidate has independent onset evidence.

25. Why convert frequency to MIDI?

MIDI provides a standard numerical pitch representation that is easy to label, compare, and visualize.

26. Why use key/scale filtering?

The estimated key provides context that can help reject short, unlikely out-of-scale artifacts. Chromatic notes still need to be preserved when appropriate.

27. Why instrument-specific polyphony limits?

Instruments differ in how many notes they commonly play simultaneously. Limits can reduce implausible detections.

28. Why check guitar playability?

A detected pitch must fit the guitar's string tuning and available fret range to be physically playable.

29. How are chords formed?

Notes with overlapping active intervals are grouped into a chord.

30. How is audio reconstructed?

The system synthesizes tones for detected pitches, combines them, and applies an amplitude envelope.

31. What are limitations?

Noise, overlapping harmonics, template mismatch, and unusual chromatic or instrumental passages can reduce accuracy. Heuristic filters can also remove a real note if its behavior resembles an artifact.

8 Last-Minute Memory Sheet

Stage	Remember it as
CQT	Musical frequency map: logarithmic pitch bins.
NMF	Template matching: W is what a note looks like; H is when it is active.
Onset detection	Finds note attacks/beginnings.
pYIN	Estimates the pitch of a mainly monophonic signal.
HMM/Viterbi	Smooths noisy frame decisions into a likely sequence.
Harmonic filter	Prevents overtones from being mistaken for extra notes.
Scale/instrument filters	Adds musical and physical context.
Event/chord stage	Turns note activity into timed notes and groups overlaps.

One sentence to remember: ChordiNote turns audio into a pitch-friendly representation, estimates note activity, finds note attacks, cleans the results using temporal and musical constraints, and outputs timed notes and chords.